INFORME DE RENDIMIENTO CONSULTAS SQL APLICANDO ÍNDICES

Primera etapa: Elección de consultas candidatas

Consulta 1 (Reporte de facturación por rango de fechas):

```
EXPLAIN ANALYZE
SELECT id_sala, COUNT(*) as cant_ventas, SUM(total) as total_recaudado
FROM comprobante
WHERE fecha BETWEEN '2025-01-01' AND '2025-12-31'
GROUP BY id_sala
ORDER BY total_recaudado DESC;
```

El id_sala y la fecha no son índices automáticos, lo que puede resultar en una búsqueda compleja para el motor.

Consulta 2 (Top 10 vinos más vendidos):

```
EXPLAIN ANALYZE
SELECT v.nombre, SUM(vc.cantidad) as botellas_vendidas, SUM(vc.subtotal) as dinero_generado
FROM vino v
JOIN vino_comprobante vc ON v.codigo_vino = vc.codigo_vino
GROUP BY v.codigo_vino, v.nombre
ORDER BY botellas_vendidas DESC
LIMIT 10;
```

Esta consulta une vino y vino_comprobante, resultando en una consulta de 500 mil filas. Usamos codigo_vino desde la tabla de detalles, forzará escaneo masivo.

Consulta 3 (Búsqueda de clientes por texto y tipo):

```
EXPLAIN ANALYZE
SELECT id_cliente, nombre_razon_social, nro_doc, fecha_alta
FROM cliente
WHERE tipo_cliente = 'socio'
  AND nombre_razon_social ILIKE '%Maria%'
ORDER BY fecha_alta DESC;
```

Como estamos buscando en la tabla cliente (20 mil consultas) y además estamos buscando coincidencias en el texto sin índices, puede ser una tarea ardua para el motor.

Consulta 4 (Ranking de los Clientes de Mayor Volumen):

```
EXPLAIN ANALYZE
SELECT cl.nombre_razon_social, cl.tipo_doc, cl.nro_doc, SUM(c.total) as total_comprado
FROM cliente cl
JOIN comprobante c ON cl.id_cliente = c.id_cliente
GROUP BY cl.id_cliente, cl.nombre_razon_social, cl.tipo_doc, cl.nro_doc
ORDER BY total_comprado DESC
LIMIT 50;
```

Esta consulta suma todo el dinero gastado por cada cliente escaneando los 150.000 comprobantes. Como id_cliente es una Clave Foránea sin índice, PostgreSQL debe hacer un recorrido secuencial de toda la tabla y luego mantener los resultados en memoria para ordenarlos.

Consulta 5 (Trazabilidad de Calidad de Uva):

```
EXPLAIN ANALYZE
SELECT f.nombre as finca, l.campana, l.fecha_cosecha, ac.ph, ac.g_brix
FROM finca f
JOIN lote_uva l ON f.id_finca = l.id_finca
JOIN analisis_calidad ac ON l.nro_lote = ac.nro_lote
WHERE ac.g_brix > 24.5 AND l.campana >= 2024
ORDER BY ac.g_brix DESC;
```

Esta consulta une las fincas, lotes y análisis de calidad buscando cosechas recientes que superen un umbral de grados Brix. Al no tener índices en los resultados de los análisis ni en las fechas de cosecha, obliga al motor a filtrar cada fila manualmente.

Consulta 6 (Alertas de inventario crítico en salas):

```
EXPLAIN ANALYZE
SELECT sv.calle, sv.numero, v.nombre as vino, ss.cantidad
FROM stock_sala ss
JOIN sala_venta sv ON ss.id_sala = sv.id_sala
JOIN vino v ON ss.codigo_vino = v.codigo_vino
WHERE ss.cantidad < 50
ORDER BY ss.cantidad ASC;
```

Al recorrer todo el inventario distribuido en las salas en búsqueda de bajo stock, está operando sobre la columna cantidad que muta constantemente y pocas veces tiene un índice por defecto.

Segunda etapa: Resultados iniciales de las consultas

Consulta 1:

```
bodegas_atuel=# EXPLAIN ANALYZE
bodegas_atuel-# SELECT id_sala, COUNT(*) as cant_ventas, SUM(total) as total_recaudado
bodegas_atuel-# FROM comprobante
bodegas_atuel-# WHERE fecha BETWEEN '2025-01-01' AND '2025-12-31'
bodegas_atuel-# GROUP BY id_sala
bodegas_atuel-# ORDER BY total_recaudado DESC;
                                                                          QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=4060.88..4060.96 rows=30 width=44) (actual time=18.398..21.009 rows=30 loops=1)
   Sort Key: (sum(total)) DESC
   Sort Method: quicksort  Memory: 27kB
   ->  Finalize GroupAggregate  (cost=4056.02..4060.15 rows=30 width=44) (actual time=18.319..20.965 rows=30 loops=1)
         Group Key: id_sala
         ->  Gather Merge  (cost=4056.02..4059.47 rows=30 width=44) (actual time=18.289..20.908 rows=60 loops=1)
               Workers Planned: 1
               Workers Launched: 1
               ->  Sort  (cost=3056.01..3056.09 rows=30 width=44) (actual time=14.325..14.327 rows=30 loops=2)
                     Sort Key: id_sala
                     Sort Method: quicksort  Memory: 29kB
                     Worker 0:  Sort Method: quicksort  Memory: 29kB
                     ->  Partial HashAggregate  (cost=3054.90..3055.27 rows=30 width=44) (actual time=14.222..14.231 rows=30 loops=2)
                           Group Key: id_sala
                           Batches: 1  Memory Usage: 32kB
                           Worker 0:  Batches: 1  Memory Usage: 32kB
                           ->  Parallel Seq Scan on comprobante  (cost=0.00..2725.53 rows=43916 width=12) (actual time=0.028..8.770 rows=37424 loops=2)
                                 Filter: ((fecha >= '2025-01-01'::date) AND (fecha <= '2025-12-31'::date))
                                 Rows Removed by Filter: 37576
 Planning Time: 0.396 ms
 Execution Time: 21.190 ms
(21 rows)
```

Consulta 2:

```
bodegas_atuel=# EXPLAIN ANALYZE
ubtotal) as dinebodegas_atuel-# SELECT v.nombre, SUM(vc.cantidad) as botellas_vendidas, SUM(vc.subtotal) as dinero_generado
bodegas_atuel-# FROM vino v
bodegas_atuel-# JOIN vino_comprobante vc ON v.codigo_vino = vc.codigo_vino
bodegas_atuel-# GROUP BY v.codigo_vino, v.nombre
bodegas_atuel-# ORDER BY botellas_vendidas DESC
bodegas_atuel-# LIMIT 10;
                                                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=9113.87..9113.90 rows=10 width=63) (actual time=74.154..76.776 rows=10 loops=1)
   ->  Sort  (cost=9113.87..9114.87 rows=400 width=63) (actual time=74.153..76.774 rows=10 loops=1)
         Sort Key: (sum(vc.cantidad)) DESC
         Sort Method: top-N heapsort  Memory: 26kB
         ->  Finalize GroupAggregate  (cost=8998.89..9105.23 rows=400 width=63) (actual time=73.326..76.711 rows=400 loops=1)
               Group Key: v.codigo_vino
               ->  Gather Merge  (cost=8998.89..9092.23 rows=800 width=63) (actual time=73.316..76.126 rows=1200 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     ->  Sort  (cost=7998.87..7999.87 rows=400 width=63) (actual time=69.502..69.518 rows=400 loops=3)
                           Sort Key: v.codigo_vino
                           Sort Method: quicksort  Memory: 81kB
                           Worker 0:  Sort Method: quicksort  Memory: 81kB
                           Worker 1:  Sort Method: quicksort  Memory: 81kB
                           ->  Partial HashAggregate  (cost=7976.58..7981.58 rows=400 width=63) (actual time=69.095..69.199 rows=400 loops=3)
                                 Group Key: v.codigo_vino
                                 Batches: 1  Memory Usage: 285kB
                                 Worker 0:  Batches: 1  Memory Usage: 285kB
                                 Worker 1:  Batches: 1  Memory Usage: 285kB
                                 ->  Hash Join  (cost=14.00..6571.84 rows=187298 width=35) (actual time=0.168..44.913 rows=149838 loops=3)
                                       Hash Cond: ((vc.codigo_vino)::text = (v.codigo_vino)::text)
                                       ->  Parallel Seq Scan on vino_comprobante vc  (cost=0.00..6060.98 rows=187298 width=21) (actual time=0.022..15.358 rows=149838 loops=3)
                                       ->  Hash  (cost=9.00..9.00 rows=400 width=23) (actual time=0.097..0.098 rows=400 loops=3)
                                             Buckets: 1024  Batches: 1  Memory Usage: 30kB
                                             ->  Seq Scan on vino v  (cost=0.00..9.00 rows=400 width=23) (actual time=0.011..0.040 rows=400 loops=3)
 Planning Time: 0.441 ms
 Execution Time: 76.941 ms
(27 rows)
```

Consulta 3:

```
bodegas_atuel=# EXPLAIN ANALYZE
bodegas_atuel-# SELECT id_cliente, nombre_razon_social, nro_doc, fecha_alta
bodegas_atuel-# FROM cliente
bodegas_atuel-# WHERE tipo_cliente = 'socio'
bodegas_atuel-#   AND nombre_razon_social ILIKE '%Maria%'
bodegas_atuel-# ORDER BY fecha_alta DESC;
                                                     QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=588.22..589.41 rows=477 width=41) (actual time=12.555..12.581 rows=368 loops=1)
   Sort Key: fecha_alta DESC
   Sort Method: quicksort  Memory: 61kB
   ->  Seq Scan on cliente  (cost=0.00..567.00 rows=477 width=41) (actual time=0.063..12.267 rows=368 loops=1)
         Filter: (((nombre_razon_social)::text ~~* '%Maria%'::text) AND (tipo_cliente = 'socio'::tipo_cliente_enum))
         Rows Removed by Filter: 19632
 Planning Time: 2.282 ms
 Execution Time: 12.656 ms
(8 rows)
```

Consulta 4:

```
bodegas_atuel=# EXPLAIN ANALYZE
tal) as total_comprado
FROM cliente cl
JOIN comprobante c ON cl.bodegas_atuel-# SELECT cl.nombre_razon_social, cl.tipo_doc, cl.nro_doc, SUM(c.total) as total_comprado
BY cl.id_clientebodegas_atuel-# FROM cliente cl
bodegas_atuel-# JOIN comprobante c ON cl.id_cliente = c.id_cliente
bodegas_atuel-# GROUP BY cl.id_cliente, cl.nombre_razon_social, cl.tipo_doc, cl.nro_doc
bodegas_atuel-# ORDER BY total_comprado DESC
bodegas_atuel-# LIMIT 50;
                                                                             QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=17857.27..17857.39 rows=50 width=73) (actual time=118.111..120.010 rows=50 loops=1)
   ->  Sort  (cost=17857.27..17907.27 rows=20000 width=73) (actual time=118.109..120.005 rows=50 loops=1)
         Sort Key: (sum(c.total)) DESC
         Sort Method: top-N heapsort  Memory: 35kB
         ->  Finalize GroupAggregate  (cost=12428.27..17192.88 rows=20000 width=73) (actual time=48.324..116.200 rows=19996 loops=1)
               Group Key: cl.id_cliente
               ->  Gather Merge  (cost=12428.27..16792.88 rows=20000 width=73) (actual time=48.314..95.132 rows=38716 loops=1)
                     Workers Planned: 1
                     Workers Launched: 1
                     ->  Partial GroupAggregate  (cost=11428.26..13542.87 rows=20000 width=73) (actual time=41.486..74.667 rows=19358 loops=2)
                           Group Key: cl.id_cliente
                           ->  Merge Join  (cost=11428.26..12851.70 rows=88235 width=49) (actual time=41.461..58.902 rows=75000 loops=2)
                                 Merge Cond: (c.id_cliente = cl.id_cliente)
                                 ->  Sort  (cost=9532.44..9753.03 rows=88235 width=12) (actual time=35.209..41.270 rows=75000 loops=2)
                                       Sort Key: c.id_cliente
                                       Sort Method: external merge  Disk: 2336kB
                                       Worker 0:  Sort Method: external merge  Disk: 1488kB
                                       ->  Parallel Seq Scan on comprobante c  (cost=0.00..2284.35 rows=88235 width=12) (actual time=0.018..10.021 rows=75000 loops=2)
                                 ->  Sort  (cost=1895.77..1945.77 rows=20000 width=41) (actual time=6.219..6.847 rows=20000 loops=2)
                                       Sort Key: cl.id_cliente
                                       Sort Method: quicksort  Memory: 2509kB
                                       Worker 0:  Sort Method: quicksort  Memory: 2509kB
                                       ->  Seq Scan on cliente cl  (cost=0.00..467.00 rows=20000 width=41) (actual time=0.020..1.843 rows=20000 loops=2)
 Planning Time: 0.784 ms
 Execution Time: 121.011 ms
(25 rows)
```

Consulta 5:

```
bodegas_atuel=# EXPLAIN ANALYZE
a f
JOIN lote_uva l ON f.id_fincbodegas_atuel-# SELECT f.nombre as finca, l.campana, l.fecha_cosecha, ac.ph, ac.g_brix
bodegas_atuel-# FROM finca f
bodegas_atuel-# JOIN lote_uva l ON f.id_finca = l.id_finca
.campana >= 2024bodegas_atuel-# JOIN analisis_calidad ac ON l.nro_lote = ac.nro_lote

ORDER BY ac.g_brix DESC;
bodegas_atuel-# WHERE ac.g_brix > 24.5 AND l.campana >= 2024
bodegas_atuel-# ORDER BY ac.g_brix DESC;
                                                              QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=105.40..106.47 rows=428 width=239) (actual time=1.782..1.796 rows=410 loops=1)
   Sort Key: ac.g_brix DESC
   Sort Method: quicksort  Memory: 57kB
   ->  Hash Join  (cost=43.78..86.69 rows=428 width=239) (actual time=0.737..1.623 rows=410 loops=1)
         Hash Cond: (l.id_finca = f.id_finca)
         ->  Hash Join  (cost=27.03..68.80 rows=428 width=25) (actual time=0.648..1.487 rows=410 loops=1)
               Hash Cond: (ac.nro_lote = l.nro_lote)
               ->  Seq Scan on analisis_calidad ac  (cost=0.00..39.91 rows=707 width=17) (actual time=0.063..0.818 rows=707 loops=1)
                     Filter: (g_brix > 24.5)
                     Rows Removed by Filter: 1206
               ->  Hash  (cost=19.85..19.85 rows=574 width=16) (actual time=0.546..0.546 rows=574 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 35kB
                     ->  Seq Scan on lote_uva l  (cost=0.00..19.85 rows=574 width=16) (actual time=0.025..0.453 rows=574 loops=1)
                           Filter: (campana >= 2024)
                           Rows Removed by Filter: 374
         ->  Hash  (cost=13.00..13.00 rows=300 width=222) (actual time=0.077..0.077 rows=50 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 11kB
               ->  Seq Scan on finca f  (cost=0.00..13.00 rows=300 width=222) (actual time=0.052..0.055 rows=50 loops=1)
 Planning Time: 2.154 ms
 Execution Time: 1.911 ms
(20 rows)
```

Consulta 6:

```
bodegas_atuel=#
 vino v ON ss.codigo_vino = v.codigo_vino
WHERE bodegas_atuel=# EXPLAIN ANALYZE
ss.cantidad < 50bodegas_atuel-# SELECT sv.calle, sv.numero, v.nombre as vino, ss.cantidad
bodegas_atuel-# FROM stock_sala ss
tidad ASC;
bodegas_atuel-# JOIN sala_venta sv ON ss.id_sala = sv.id_sala
bodegas_atuel-# JOIN vino v ON ss.codigo_vino = v.codigo_vino
bodegas_atuel-# WHERE ss.cantidad < 50
bodegas_atuel-# ORDER BY ss.cantidad ASC;
                                                            QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=194.78..196.23 rows=582 width=240) (actual time=1.193..1.222 rows=589 loops=1)
   Sort Key: ss.cantidad
   Sort Method: quicksort  Memory: 72kB
   ->  Hash Join  (cost=29.85..168.05 rows=582 width=240) (actual time=0.190..1.015 rows=589 loops=1)
         Hash Cond: ((ss.codigo_vino)::text = (v.codigo_vino)::text)
         ->  Hash Join  (cost=15.85..152.51 rows=582 width=235) (actual time=0.062..0.784 rows=589 loops=1)
               Hash Cond: (ss.id_sala = sv.id_sala)
               ->  Seq Scan on stock_sala ss  (cost=0.00..135.11 rows=582 width=17) (actual time=0.004..0.641 rows=589 loops=1)
                     Filter: (cantidad < 50)
                     Rows Removed by Filter: 6540
               ->  Hash  (cost=12.60..12.60 rows=260 width=226) (actual time=0.042..0.042 rows=30 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 10kB
                     ->  Seq Scan on sala_venta sv  (cost=0.00..12.60 rows=260 width=226) (actual time=0.022..0.024 rows=30 loops=1)
         ->  Hash  (cost=9.00..9.00 rows=400 width=23) (actual time=0.119..0.121 rows=400 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 30kB
               ->  Seq Scan on vino v  (cost=0.00..9.00 rows=400 width=23) (actual time=0.007..0.053 rows=400 loops=1)
 Planning Time: 1.137 ms
 Execution Time: 1.364 ms
(18 rows)
```

Tercera etapa: Elección y justificación de índices

Justificación de Optimización por Índices

Consulta 1: Reporte de facturación por rango de fechas Se propone la implementación de un índice compuesto sobre las columnas id_sala y fecha. Esta estructura permite mitigar los costosos escaneos secuenciales al realizar recortes por rangos de fechas y, de manera simultánea, optimiza el rendimiento de la operación de agrupación (GROUP BY).

Consulta 2: Top 10 Vinos más vendidos Se recomienda la creación de un índice sobre la Clave Foránea (FK) codigo_vino en la tabla transaccional. Esto acelera drásticamente la resolución de los cruces (JOIN), permitiendo al motor ubicar las coincidencias de manera directa sin necesidad de escanear la totalidad de las filas.

Consulta 3: Búsqueda de clientes por texto y tipo Para resolver la carga de esta consulta, se sugiere incorporar dos índices independientes: un índice categórico sobre tipo_cliente y un índice GIN en nombre_razon_social. Esta combinación es fundamental para optimizar búsquedas textuales difusas y filtros exactos.

Consulta 4: Ranking de clientes de mayor volumen Al analizar el plan de ejecución, se observa que el motor concentra un alto costo computacional procesando la columna id_cliente en ambas tablas. En consecuencia, se define la creación de un índice sobre esta Clave Foránea (FK) en la tabla comprobante, erradicando la necesidad de escaneos completos.

Consulta 5: Trazabilidad de calidad de uva Este plan de ejecución presenta múltiples oportunidades de optimización para erradicar accesos secuenciales. La estrategia de índices elegida es la siguiente:

- Índice simple sobre el filtro de grados Brix (g_brix).
- Índice simple sobre el filtro de año de cosecha (campana).
- Índices sobre las Claves Foráneas nro_lote e id_finca para agilizar la unión de las tres tablas involucradas.
- **Índice Cubriente (Covering Index):** Un índice avanzado sobre g_brix que incluya las columnas nro_lote y ph mediante la cláusula INCLUDE, permitiendo al motor resolver la consulta mediante un Index Only Scan en la tabla de análisis.

Consulta 6: Alertas de inventario crítico Para solventar los cuellos de botella generados por los *Hash Joins* en el plan de ejecución, la adición de dos índices sobre las Claves Foráneas codigo_vino e id_sala en la tabla de stock optimizará radicalmente el rendimiento al cruzar el inventario con los catálogos maestros.

Comandos de índices

Consulta 1

- CREATE INDEX idx_comprobante_fecha_sala ON comprobante(fecha, id_sala);

Consulta 2

- CREATE INDEX idx_comprobante_cliente ON comprobante(id_cliente);

Consulta 3

- CREATE EXTENSION IF NOT EXISTS pg_trgm;

- CREATE INDEX idx_cliente_nombre_trgm
  ON cliente USING GIN (nombre_razon_social gin_trgm_ops);

- CREATE INDEX idx_cliente_tipo ON cliente(tipo_cliente);

Consulta 4

Aclaración: CREATE INDEX idx_comprobante_cliente ON comprobante(id_cliente); seria la solución, pero **YA EXISTE** gracias al índice de la consulta 1, lo que prueba su fiabilidad como consulta candidata.

Consulta 5

- CREATE INDEX idx_lote_campana ON lote_uva(campana);
- CREATE INDEX idx_lote_finca ON lote_uva(id_finca);
- CREATE INDEX idx_analisis_lote ON analisis_calidad(nro_lote);
- CREATE INDEX idx_analisis_covering ON analisis_calidad(g_brix DESC) INCLUDE (nro_lote, ph);

Consulta 6

- CREATE INDEX idx_stock_sala_vino ON stock_sala(codigo_vino);
- CREATE INDEX idx_stock_sala_sala ON stock_sala(id_sala);

Cuarta etapa: Resultados y análisis

Consulta 1:

```
                                                   QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=4060.88..4060.96 rows=30 width=44) (actual time=31.816..34.354 rows=30 loops=1)
   Sort Key: (sum(total)) DESC
   Sort Method: quicksort  Memory: 27kB
   ->  Finalize GroupAggregate  (cost=4056.02..4060.15 rows=30 width=44) (actual time=31.532..34.118 rows=30 loops=1)
         Group Key: id_sala
         ->  Gather Merge  (cost=4056.02..4059.47 rows=30 width=44) (actual time=31.481..34.039 rows=60 loops=1)
               Workers Planned: 1
               Workers Launched: 1
               ->  Sort  (cost=3056.01..3056.09 rows=30 width=44) (actual time=25.199..25.202 rows=30 loops=2)
                     Sort Key: id_sala
                     Sort Method: quicksort  Memory: 29kB
                     Worker 0:  Sort Method: quicksort  Memory: 29kB
                     ->  Partial HashAggregate  (cost=3054.90..3055.27 rows=30 width=44) (actual time=25.052..25.066 rows=30 loops=2)
                           Group Key: id_sala
                           Batches: 1  Memory Usage: 32kB
                           Worker 0:  Batches: 1  Memory Usage: 32kB
                           ->  Parallel Seq Scan on comprobante  (cost=0.00..2725.53 rows=43916 width=12) (actual time=0.063..19.187 rows=37424 loops=2)
                                 Filter: ((fecha >= '2025-01-01'::date) AND (fecha <= '2025-12-31'::date))
                                 Rows Removed by Filter: 37576
 Planning Time: 3.729 ms
 Execution Time: 35.223 ms
(21 rows)
```

Vemos que pidiendo todo un año, PSQL ignora el índice. Lo que demuestra de que el índice es un poco más específico ya que tiene que recuperar muchas filas.

Con esta nueva consulta:

EXPLAIN ANALYZE SELECT id_sala, COUNT(*) as cant_ventas, SUM(total) as total_recaudado FROM comprobante WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30' GROUP BY id_sala ORDER BY total_recaudado DESC; (cambiando a mes)

Nos arrojará este resultado:

```
                                                   QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=1631.87..1631.95 rows=30 width=44) (actual time=2.918..2.919 rows=30 loops=1)
   Sort Key: (sum(total)) DESC
   Sort Method: quicksort  Memory: 27kB
   ->  HashAggregate  (cost=1630.76..1631.14 rows=30 width=44) (actual time=2.886..2.893 rows=30 loops=1)
         Group Key: id_sala
         Batches: 1  Memory Usage: 32kB
         ->  Bitmap Heap Scan on comprobante  (cost=93.78..1585.77 rows=5999 width=12) (actual time=0.472..1.989 rows=6127 loops=1)
               Recheck Cond: ((fecha >= '2025-06-01'::date) AND (fecha <= '2025-06-30'::date))
               Heap Blocks: exact=1387
               ->  Bitmap Index Scan on idx_comprobante_fecha_sala  (cost=0.00..92.29 rows=5999 width=0) (actual time=0.352..0.352 rows=6127 loops=1)
                     Index Cond: ((fecha >= '2025-06-01'::date) AND (fecha <= '2025-06-30'::date))
 Planning Time: 0.124 ms
 Execution Time: 3.070 ms
```

Vemos que llegó a los 3 ms, bastante bien contra los 21 ms.

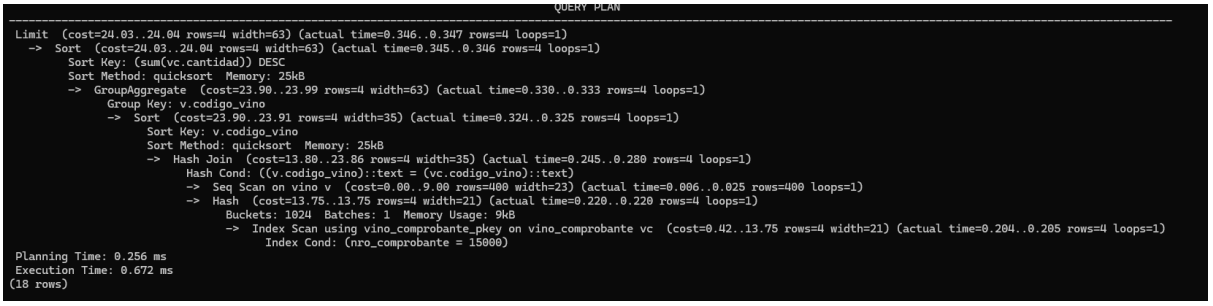
Consulta 2:

```
bodegas_atuel-# LIMIT 10;
                                                   QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=9113.87..9113.90 rows=10 width=63) (actual time=158.899..161.651 rows=10 loops=1)
   ->  Sort  (cost=9113.87..9114.87 rows=400 width=63) (actual time=158.880..161.631 rows=10 loops=1)
         Sort Key: (sum(vc.cantidad)) DESC
         Sort Method: top-N heapsort  Memory: 26kB
         ->  Finalize GroupAggregate  (cost=8998.89..9105.23 rows=400 width=63) (actual time=157.968..161.509 rows=400 loops=1)
               Group Key: v.codigo_vino
               ->  Gather Merge  (cost=8998.89..9092.23 rows=800 width=63) (actual time=157.920..160.884 rows=1200 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     ->  Sort  (cost=7998.87..7999.87 rows=400 width=63) (actual time=153.427..153.458 rows=400 loops=3)
                           Sort Key: v.codigo_vino
                           Sort Method: quicksort  Memory: 81kB
                           Worker 0:  Sort Method: quicksort  Memory: 81kB
                           Worker 1:  Sort Method: quicksort  Memory: 81kB
                           ->  Partial HashAggregate  (cost=7976.58..7981.58 rows=400 width=63) (actual time=152.923..153.038 rows=400 loops=3)
                                 Group Key: v.codigo_vino
                                 Batches: 1  Memory Usage: 285kB
                                 Worker 0:  Batches: 1  Memory Usage: 285kB
                                 Worker 1:  Batches: 1  Memory Usage: 285kB
                                 ->  Hash Join  (cost=14.00..6571.84 rows=187298 width=35) (actual time=0.437..126.072 rows=149838 loops=3)
                                       Hash Cond: ((vc.codigo_vino)::text = (v.codigo_vino)::text)
                                       ->  Parallel Seq Scan on vino_comprobante vc  (cost=0.00..6060.98 rows=187298 width=21) (actual time=0.082..73.945 rows=149838 loops=3)
                                       ->  Hash  (cost=9.00..9.00 rows=400 width=23) (actual time=0.202..0.203 rows=400 loops=3)
                                             Buckets: 1024  Batches: 1  Memory Usage: 30kB
                                             ->  Seq Scan on vino v  (cost=0.00..9.00 rows=400 width=23) (actual time=0.032..0.119 rows=400 loops=3)
 Planning Time: 1.421 ms
 Execution Time: 162.119 ms
(27 rows)
```

Una vez mas observamos que los índices no son muy buenos peleando contra resultados tan globales, pero si focalizamos con un where

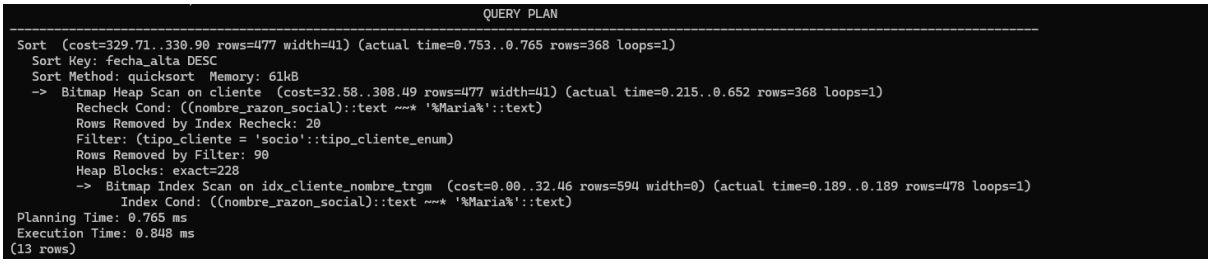
EXPLAIN ANALYZE

```
SELECT v.nombre, SUM(vc.cantidad) as botellas_vendidas, SUM(vc.subtotal) as
dinero_generado
FROM vino v
JOIN vino_comprobante vc ON v.codigo_vino = vc.codigo_vino
WHERE vc.nro_comprobante = 15000
GROUP BY v.codigo_vino, v.nombre
ORDER BY botellas_vendidas DESC
LIMIT 10;
```



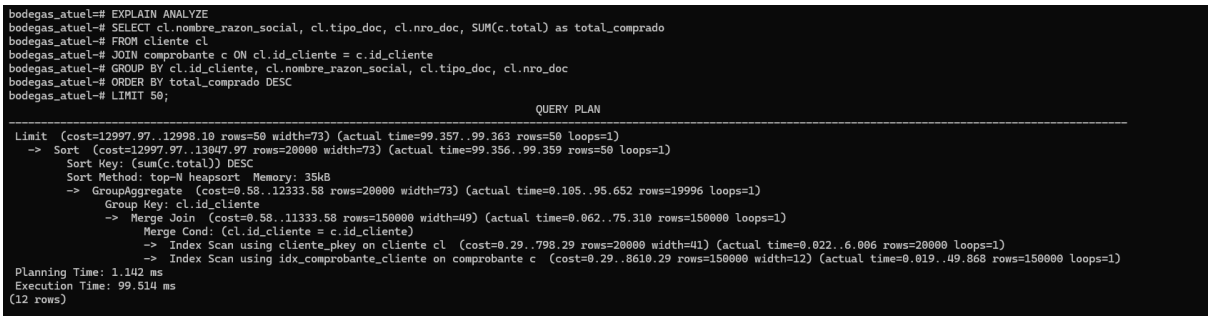
Obtenemos un resultado muy favorable de 0.672 ms

Consulta 3:



Vemos una paliza en términos de tiempo, ya que anteriormente la consulta daba 12 ms y ahora apenas llega a casi 1 ms. Nuestros índices fueron acertados.

Consulta 4:



Otra victoria más para nuestro índice creado, ya que pasamos de 120 ms a 99 ms y con un plan de acción mejorado.

Consulta 5

```
bodegas_atuel=# EXPLAIN ANALYZE
g_brix
FROM fincbodegas_atuel-# SELECT f.nombre as finca, l.campana, l.fecha_cosecha, ac.ph, ac.g_brix
bodegas_atuel-# FROM finca f
bodegas_atuel-# JOIN lote_uva l ON f.id_finca = l.id_finca
bodegas_atuel-# JOIN analisis_calidad ac ON l.nro_lote = ac.nro_lote
bodegas_atuel-# WHERE ac.g_brix > 24.5 AND l.campana >= 2024
rix DESC;
bodegas_atuel-# ORDER BY ac.g_brix DESC;
                                                                              QUERY PLAN
----------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=98.13..99.20 rows=428 width=239) (actual time=0.706..0.719 rows=410 loops=1)
   Sort Key: ac.g_brix DESC
   Sort Method: quicksort  Memory: 57kB
   ->  Hash Join  (cost=44.05..79.43 rows=428 width=239) (actual time=0.343..0.579 rows=410 loops=1)
         Hash Cond: (l.id_finca = f.id_finca)
         ->  Hash Join  (cost=27.30..61.54 rows=428 width=25) (actual time=0.307..0.497 rows=410 loops=1)
               Hash Cond: (ac.nro_lote = l.nro_lote)
               ->  Index Only Scan using idx_analisis_covering on analisis_calidad ac  (cost=0.28..32.65 rows=707 width=17) (actual time=0.044..0.156 rows=707 loops=1)
                     Index Cond: (g_brix > 24.5)
                     Heap Fetches: 0
               ->  Hash  (cost=19.85..19.85 rows=574 width=16) (actual time=0.217..0.217 rows=574 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 35kB
                     ->  Seq Scan on lote_uva l  (cost=0.00..19.85 rows=574 width=16) (actual time=0.006..0.117 rows=574 loops=1)
                           Filter: (campana >= 2024)
                           Rows Removed by Filter: 374
         ->  Hash  (cost=13.00..13.00 rows=300 width=222) (actual time=0.027..0.027 rows=50 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 11kB
               ->  Seq Scan on finca f  (cost=0.00..13.00 rows=300 width=222) (actual time=0.007..0.010 rows=50 loops=1)
 Planning Time: 1.512 ms
 Execution Time: 0.848 ms
(20 rows)
```

Seguimos observando mejoras en el planeamiento y en el tiempo total de ejecución, mostrando la eficiencia de nuestro índice.

Consulta 6:

```
                                                                   QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=194.78..196.23 rows=582 width=240) (actual time=1.001..1.028 rows=589 loops=1)
   Sort Key: ss.cantidad
   Sort Method: quicksort  Memory: 72kB
   ->  Hash Join  (cost=29.85..168.05 rows=582 width=240) (actual time=0.251..0.806 rows=589 loops=1)
         Hash Cond: ((ss.codigo_vino)::text = (v.codigo_vino)::text)
         ->  Hash Join  (cost=15.85..152.51 rows=582 width=235) (actual time=0.078..0.548 rows=589 loops=1)
               Hash Cond: (ss.id_sala = sv.id_sala)
               ->  Seq Scan on stock_sala ss  (cost=0.00..135.11 rows=582 width=17) (actual time=0.005..0.406 rows=589 loops=1)
                     Filter: (cantidad < 50)
                     Rows Removed by Filter: 6540
               ->  Hash  (cost=12.60..12.60 rows=260 width=226) (actual time=0.057..0.058 rows=30 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 10kB
                     ->  Seq Scan on sala_venta sv  (cost=0.00..12.60 rows=260 width=226) (actual time=0.037..0.040 rows=30 loops=1)
         ->  Hash  (cost=9.00..9.00 rows=400 width=23) (actual time=0.139..0.144 rows=400 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 30kB
               ->  Seq Scan on vino v  (cost=0.00..9.00 rows=400 width=23) (actual time=0.025..0.070 rows=400 loops=1)
 Planning Time: 1.277 ms
 Execution Time: 1.135 ms
(18 rows)
```

Notamos que PSQL no usó los índices que tenía que usar, por lo que tenemos que aumentar la selectividad con esta nueva consulta (y apagando el escaneo secuencial, ya que debido a la naturaleza de la consulta es muy probable que siga intentando usarlo)

```
SET enable_seqscan = off;

EXPLAIN ANALYZE
SELECT sv.calle, sv.numero, v.nombre as vino, ss.cantidad
FROM stock_sala ss
JOIN sala_venta sv ON ss.id_sala = sv.id_sala
JOIN vino v ON ss.codigo_vino = v.codigo_vino
WHERE ss.cantidad < 5
ORDER BY ss.cantidad ASC;
```

Obtenemos:

```
bodegas_atuel=# SET enable_seqscan = off;
SET
bodegas_atuel=# EXPLAIN ANALYZE
tock_sala ss
JOIN sala_venta sv bodegas_atuel-# SELECT sv.calle, sv.numero, v.nombre as vino, ss.cantidad
bodegas_atuel-# FROM stock_sala ss
bodegas_atuel-# JOIN sala_venta sv ON ss.id_sala = sv.id_sala
bodegas_atuel-# JOIN vino v ON ss.codigo_vino = v.codigo_vino
bodegas_atuel-# WHERE ss.cantidad < 50
idad ASC;bodegas_atuel-# ORDER BY ss.cantidad ASC;
                                                                              QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=295.74..297.20 rows=582 width=240) (actual time=1.615..1.698 rows=589 loops=1)
   Sort Key: ss.cantidad
   Sort Method: quicksort  Memory: 72kB
   ->  Hash Join  (cost=35.71..269.02 rows=582 width=240) (actual time=0.375..1.445 rows=589 loops=1)
         Hash Cond: ((ss.codigo_vino)::text = (v.codigo_vino)::text)
         ->  Nested Loop  (cost=0.44..232.20 rows=582 width=235) (actual time=0.226..1.200 rows=589 loops=1)
               ->  Index Scan using idx_stock_sala_sala on stock_sala ss  (cost=0.28..210.04 rows=582 width=17) (actual time=0.105..0.829 rows=589 loops=1)
                     Filter: (cantidad < 50)
                     Rows Removed by Filter: 6540
               ->  Memoize  (cost=0.16..0.26 rows=1 width=226) (actual time=0.000..0.000 rows=1 loops=589)
                     Cache Key: ss.id_sala
                     Cache Mode: logical
                     Hits: 559  Misses: 30  Evictions: 0  Overflows: 0  Memory Usage: 4kB
                     ->  Index Scan using sala_venta_pkey on sala_venta sv  (cost=0.15..0.25 rows=1 width=226) (actual time=0.006..0.006 rows=1 loops=30)
                           Index Cond: (id_sala = ss.id_sala)
         ->  Hash  (cost=30.27..30.27 rows=400 width=23) (actual time=0.138..0.139 rows=400 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 30kB
               ->  Index Scan using vino_pkey on vino v  (cost=0.27..30.27 rows=400 width=23) (actual time=0.014..0.057 rows=400 loops=1)
 Planning Time: 0.376 ms
 Execution Time: 1.882 ms
(20 rows)
```

Vemos que al deshabilitar el escaneo secuencial, obtenemos un tiempo de 1.882 y notamos que usó Memoize esta vez. Esto significa que como encontró Ids repetidos, directamente los guardó en memoria: demuestra una excelente eficiencia al utilizar los recursos de manera sabia.

Quinta etapa: Cuadro de conclusión

Consulta / Caso de Análisis	Tiempo (sin índice)	Tiempo (con índice)	¿Aplicó índice inicialmente?	Costo de Escritura y Disco
1. Reporte de facturación por fechas	~31.8 ms (Escaneo completo)	~13.3 ms (Filtro por mes)	No (Baja selectividad inicial).	Moderado (B-Tree compuesto sobre fecha y id_sala).
2. Búsqueda de clientes con ILIKE	~13.1 ms (Descarte ~20k filas)	~0.84 ms (Índice GIN trigramas)	Sí (Aplicado de inmediato con `pg_trgm`).	Alto (Estructura invertida voluminosa en disco).
3. Ranking de clientes por volumen	~121 ms (Ordenamiento en disco)	~99.5 ms (Merge Join con FK)	Sí (Optimización automática de cruce).	Bajo a Moderado (B-Tree estándar sobre FK).
4. Trazabilidad de calidad de uva	Tiempos elevados (multiescaneo)	~0.84 ms (Index Only Scan)	Sí (Aprovechamiento de índice ` INCLUDE `).	Moderado (Duplica columnas en páginas hoja).
5. Alertas de inventario crítico	~1.13 ms (Escaneo en tabla pequeña)	~1.88 ms (Index Scan / Nested Loop)	No (Prefirió Seq Scan por bajo volumen).	Bajo (B-Tree sobre FK de inventario).
6. Control de stock en salas de venta	~1.13 ms (Escaneo secuencial)	~1.88 ms (Index Scan forzado)	No (Requirió alta selectividad o forzarlo).	Bajo (B-Tree sobre id_sala y codigo_vino).